

2026 年 AI 编码的“渐进式 Spec”实战指南

原创 逸驹 阿里云开发者 2026年4月2日 08:31 浙江



阿里妹导读



这次分享的内容来自作者在实际项目中落地 AI 编码的一些实践和思考。希望能给正在尝试或想要尝试 AI 编码的同学一些参考。

一、背景

聊 AI 编码之前，先对齐三个基础认知

1.1 如何理解大模型 — 它能做什么、不能做什么

当前顶级模型可以独立完成中等复杂度的编码任务——理解需求、读代码、写实现、修编译错误，但仍需人审查结果。它们没有持久记忆、没有自主意图，只处理你给它的上下文。

不同模型之间的性能差异是断崖式的。Chatbot Arena 通过真人盲评计算 ELO 分数（截至 2026 年 3 月，536 万次投票、316 个模型）：

梯队	代表模型	Overall	Coding	说明
T0	Claude 4.6 Opus、Gemini 3.1 Pro	1-3	1-3	综合和编码均领先
T1	Grok 4.20、GPT 5.2、Gemini 3 Pro、Claude Sonnet 4.6	4-8	4-8	主力级
T1.5	Dola-Seed 2.0、Kimi-K2.5（月之暗面）、GLM-5（智谱）	10-16	7-12	国产前沿，Coding 已进前 12
T2	Claude 4.5 Sonnet（上代）、GPT 5.1、Gemini 3 Flash	14-16	8-16	日常够用

Arena 的 **Multi-Turn** 维度衡量多轮交互稳定性（对应 Agent 场景）：Gemini 3.1 Pro 多轮第一，超过 Claude 4.6；Claude Sonnet 4.6 掉到第 10——单轮强不等于多轮稳。

梯队差异体感：同样是“给 Spring Boot 服务加个带缓存的分页查询”——T0 一次生成全链路且主动处理边界情况；T1 多提示一两轮可达到接近效果；T1.5 基本可用但容易漏边界；T2 能写骨架但需要较多人工调整。差距不在“能不能写”，而在一次做对的概率——T0 三轮搞定的事，T2 可能 15 轮还不一定对。

核心结论：模型是地基，方法论是上层建筑。地基不行，上面盖得再好也白搭。

1.2 如何理解 Agent — 从一问一答到自主行动

知道了模型能力之后，下一个问题是：怎么让它自主行动？

裸 LLM 只是一个无状态的问答函数——你问一句它答一句，没有工具、没有记忆、不会自己行动。

Agent = while 循环 + Tool Use + 工具执行器，用一个例子说明：

1	你说：“把 UserService 里的 getById 方法加个缓存”	
2	→ Agent 调用【读文件】工具，读取 UserService.java	（侦察）
3	→ Agent 分析代码，决定修改方案	（思考）
4	→ Agent 调用【写文件】工具，插入缓存逻辑	（行动）
5	→ Agent 调用【终端】工具，运行编译检查	（验证）
6	→ 编译报错，Agent 读取错误信息，自动修复	（自愈）
7	→ 编译通过，Agent 回复你“已完成”	（结束）

这个循环就是 Agent 的全部—— "智能"来自模型, "能力"来自工具, "自主性"来自循环。

关键理解: 工具的边界就是 Agent 的能力边界。给它读写文件的工具, 它能改代码; 不给它网络工具, 它就上不了网。安全靠框架约束, 不靠 AI 自觉。

我们后面提到的 Cursor、Claude Code、opencode——本质上都是这个循环的不同包装, 区别只在于给了哪些工具、跑在哪里、用的什么模型。

1.3 回归本质 — 软件复杂度视角

有了能自主行动的 Agent, 最后一个是: 用什么标准评判一个 AI 编码方案的好坏?

来自《人月神话》的核心洞察

- 软件复杂度 = 本质复杂度 (业务逻辑本身, 不可消除) + 偶然复杂度 (工具/流程引入的额外负担, 可以且应该被消除);
- 本质复杂度由业务决定, 任何工具都不能消除它——交易状态机该多复杂还是多复杂;
- AI Coding 工具能做的是帮你更高效地应对本质复杂度 (快速理解代码、生成实现、发现风险), 但工具自身也会引入偶然复杂度 (学习成本、流程开销、配置负担);
- 评判标准: 一个方案好不好, 看它能多高效地帮你应对本质复杂度, 同时自身引入的偶然复杂度有多低;
- 核心结论: 所有方法论的设计都要回归这个起点——高效应对本质复杂度, 压缩偶然复杂度。

二、渐进式编码框架

2.1 Spec Coding 是什么

一句话: 在让 AI 写代码之前, 先用结构化文档 (Spec) 把"要做什么、怎么做、有什么约束"说清楚, 然后 AI 围绕这份文档编码。

为什么需要 Spec Coding——直接和 AI 聊天写代码 (Vibe Coding) 面临四个工程问题:

问题	根因	Spec 怎么解
上下文腐烂	对话越长 AI 越忘事，第 15 轮把第 5 轮的修改覆盖回去	Spec 是外部状态锁，强制 AI 每次行动前读取，状态永久"满血"
审查瘫痪	AI 秒生成 500 行代码，人根本 Review 不过来	先审 Plan（逻辑）再写 Code，审逻辑代替审代码
维护断层	全是 AI 生成的陌生代码，两周后不敢动	文档即源码，修 Bug 先修文档，代码随时可重新生成
代码不信任	不知道 AI 为什么这么写，不敢上线	Spec + 日志 + 代码三方交叉验证，增强信心

Spec Coding 三条铁律：

- 1. No Spec, No Code — 没有文档，不准写代码
- 2. Spec is Truth — 文档和代码冲突时，错的一定是代码
- 3. Reverse Sync — 发现 Bug，先修文档，再修代码

这三条铁律在经济上也是合理的（Code is Cheap, Context is Expensive）：

Token 类型	相对价格	说明
输出 Token	100%（基准）	最贵，AI 生成的每一个字都是钱
输入 Token	~10%	喂给 AI 的上下文
缓存输入 Token	~1%	重复使用的上下文（如 Spec 文档）

把需求、约束、代码现状写进 Spec 作为高质量输入 → 输入增加但便宜 → AI 不用反复试错 → 输出大幅减少 → 对话轮次从 20 轮降到 3-5 轮 → 总成本反而更低，效果反而更好。

2.2 为什么要自己做一套

Spec Coding 的理念很简单，但不同团队的落地方式差别很大。我们调研了多个主流实现后，吸收各方核心理念做了一套自己的框架。

理念来源	吸收了什么	怎么落地的
SDD-RIPER	Spec 驱动三法则、Research 事实锁定、Plan 是合同、三角定位验证	/propose → /apply → /review 流程
Superpowers	HARD-GATE、两阶段 review、verification 铁律	确认门控 + Sub Agent 审查
Simon Willison	Code is Cheap, Context is Expensive	Spec = 高质量输入，用输入 token 换输出 token
同行实践	知识库双层结构、触发关键词按需召回、渐进式披露	knowledge/index.md + 复杂度策略

2.3 核心设计：渐进式复杂度

这是框架的核心卖点，也是和其他方案最大的区别。

为什么需要渐进式？其他方案都假设所有需求都值得走完整流程，但现实中并非如此——70% 的需求是 ≤5 人日的小需求。改个字段、修个 bug，也要先写 spec 再拆 tasks？这就是偶然复杂度在吃掉你的效率。

核心思想：不同复杂度的需求，暴露不同深度的流程——

复杂度	使用的层	典型场景	占比
● 简单	rules/ + 直接对话	改字段、修 bug、加日志	~70%
● 中等	+ spec.md	新增接口、迁移模块	~25%
● 复杂	+ spec + tasks + knowledge	链路迁移、新子系统、跨模块	~5%

关键原则：

- 简单需求不承担复杂流程的成本——改个字段不需要先写 spec 再拆 tasks

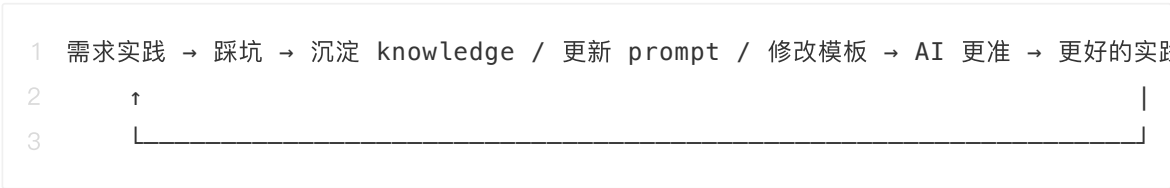
- 流程是可选增强，而非强制前提——Rules 始终生效，Spec 按复杂度加载
- 这本质上是在压缩偶然复杂度：只有本质复杂度够高时，才引入对应重量的流程

2.4 自我迭代：一切皆可迭代

这个框架本身就是一个活的系统——prompt、模板、rules 都是代码库中的普通文件，随 Git 版本演进：

可迭代的部分	文件位置	迭代触发
Agent 提示词	agents/copilot-prompt.md	AI 行为不符预期 → 修改 → 下次会话自动生效
Spec 模板	changes/templates/spec.md	发现缺少关键章节 → 修改 → 下次 /propose 自动使用
Tasks / Log 模板	changes/templates/	同上
项目规则	rules/*.md	AI 反复犯某类错误 → 加一条 rule → 全局生效
领域知识	knowledge/*.md	踩坑沉淀 → 下次 /propose 自动召回

知识飞轮（不仅是领域知识，prompt 和模板自身也在飞轮中）：



2.5 框架全貌：目录结构

```
1 code_copilot/
2 |— rules/                # Project Rules (始终生效)
3 |   |— project-context.md # 工程结构、分层、核心依赖
4 |   |— coding-style.md    # 编码规范
5 |   |— security.md        # 安全红线
6 |   └— domain-rules.md    # 业务领域约束
7 |
8 |— knowledge/            # 领域知识 (按需加载)
```

```

 9 |   |— index.md           # 知识索引 (触发关键词 + 描述)
10 |   |— *.md             # 详细知识文档
11 |
12 | — agents/             # Agent 配置与提示词
13 |   |— copilot-prompt.md # 实际提示词 (AI 动态加载, 可迭代)
14 |   |— spec-reviewer.md  # Spec 合规审查 Agent
15 |   |— code-quality-reviewer.md # 代码质量审查 Agent
16 |
17 | — changes/            # 变更管理
18 |   |— templates/       # 模板 (spec / tasks / log, 可迭代)
19 |   |— <change-name>/    # 每个需求一个目录
20 |
21 | — archives/           # 已完成变更的归档

```

Agent 提示词的核心设计要点：

1. **身份定位**：有经验的 Java 后端工程师搭档，不是代码生成器
2. **启动流程**：每次会话自动读取 rules/、检查 changes/ 进行中的变更、报告状态
3. **命令式路由**：7 个命令 (/init /propose /apply /fix /review /archive /knowledge)，超出范围礼貌拒绝
4. **Research 约束**：代码现状必须有出处（文件路径+类名/方法名），不接受空口结论
5. **执行策略**：默认逐步执行（暂停确认），支持批量和紧急停车
6. **Reverse Sync**：遇到偏差先修 spec 再修代码，强制回写
7. **安全红线**：涉及资金/状态变更 → ⚠ 高亮提醒人工审查
8. **知识沉淀**：有价值的发现 → 主动建议沉淀到 knowledge/

2.6 workflow: Propose → Apply → Review → Archive

Propose（提案）— 人主导，AI 辅助

1. **Research**：分析代码现状，锁定事实（每个结论有代码出处）
2. **逐个提问**：一次只问一个问题或一组紧密相关的问题，优先给 2-3 个选项 + 推荐，减少用户思考负担。同时做 YAGNI 裁剪——主动识别"nice to have"建议延后
3. **分段生成文档，每段确认**：不一口气生成完整 spec，按段输出（代码现状+功能点 → 变更范围+风险 → 技术决策+待澄清），每段等用户确认后再继续。越早发现方向偏差，修正成本越低

4. 生成完整 spec.md + tasks.md + log.md

5. **关键约束**：待澄清全部解决前，不允许进入 Apply

6. **确认门控 HARD-GATE**：完整 spec + tasks 生成后，必须等用户显式确认。确认前禁止任何编码动作——再简单的需求也值得一次设计审视

Apply（执行）— AI 主导，人审查

- 默认逐步执行：完成一个 task → 报告 → 等用户确认
- 批量执行：用户说"全部完成" → 按顺序执行所有
- 紧急停车：遇到逻辑冲突或 spec 缺失 → 立即停止，Reverse Sync
- **零偏差原则**：Plan 是合同，AI 是打印机
- **Verification 铁律**：每个 task 完成后必须展示可验证的证据（编译输出 / 测试输出 / 调用结果），禁止"应该没问题"等无证据声明
- **实时知识采集**：每个 task 完成后立即检查是否踩坑/发现隐含规则/学到新东西，有则立即写入 log.md

Fix（修正迭代）— Review 后的增量修正

- 填补 /apply 和 /review 之间的修正环节
- 与 /apply 的区别：/apply 按 tasks 顺序执行初始编码，/fix 在已完成基础上做增量修正
- **文档同步是铁律**——每次 /fix 必须同步更新 spec、tasks、log

Review（审查）— 两阶段 Sub Agent 审查

拆为两个独立阶段，通过 Sub Agent 执行（上下文与实现者隔离）：

1. **Spec Compliance (spec-reviewer)**：逐条比对 spec 功能点与实际代码，核心原则"不信报告只信代码"
2. **Code Quality (code-quality-reviewer)**：基于 rules/ 检查编码规范、安全红线、异常处理，按 Critical/Important/Minor 分级

阶段一 PASS 后才启动阶段二。任一 FAIL 则回到 Apply/Fix 修正。

Archive（归档）— 知识沉淀

逐条展示 log.md 中的知识发现和踩坑记录，询问用户是否沉淀到 knowledge/，确认的立即执行。变更目录移到 archives/。

Debug — 系统化调试流程

四阶段调试指引：根因调查 → 模式分析 → 假设验证 → 实施修复。**铁律：**禁止在未确认根因前直接改代码。

三、工具选型与编排

3.1 编排层 + 执行层的两层架构

在实践中，我发现单一工具很难同时满足"强模型做决策"和"安全模型写代码"两个需求。最终演化出了编排层 + 执行层的两层 AI 架构：



为什么要分两层？ 不仅仅是安全考虑，更是关注点分离：

层	擅长	模型选择
编排层	理解模糊需求、生成结构化 spec、跨仓库业务分析、审查决策	强模型（Claude Opus、Gemini Pro 等）
执行层	读写代码、执行 shell 命令、快速迭代修改	编码优化模型（Sonnet、Kimi 等）

把两者混在一起，要么模型太贵（全程用顶级模型写代码），要么质量不够（全程用便宜模型做决策）。分层后各取所长，成本和质量都更优。

3.2 工具选型思路

选择编码工具时，有一个关键经验：**透明度不是奢侈品，是基础需求。**

透明度底线：模型型号+版本可见、完整 context 可查、原始输出不被篡改、token 用量透明。

在不透明的工具上花再多时间优化 prompt 和框架，效果都无法归因、无法复现。切到透明工具链后，每次调优都能看到效果，迭代速度指数级提升。

目前满足透明度要求的主流开源工具：

工具	定位	特点
Claude Code	终端 AI 编码 Agent	Anthropic 官方，模型绑定 Claude
opencode	终端 AI 编码 Agent（开源）	模型自由选择，社区驱动
Cursor / Windsurf	IDE 内交互式 AI 搭档	GUI 友好，上手快
Cline / Aider	终端/IDE 插件	轻量级，可定制

选择建议：

- 如果团队用 Claude 系列模型，Claude Code 开箱即用
- 如果需要灵活切换模型（特别是接入私有部署的模型），opencode 更合适
- 如果团队更习惯 IDE 工作流，Cursor/Windsurf 是好选择
- **核心原则：**工具是手段，方法论是不变的——上面的框架（rules/ + knowledge/ + changes/）可以适配任何编码工具

3.3 Git 规范

无论用什么工具，Git 规范是必须的：

1. **禁止 master 分支变更：**编码前检查当前分支，master 上立即停止
2. **自动 Commit：**每个 task/fix 完成后自动 commit，保持一个 task 一个 commit
3. **Commit 必须可编译：**commit 前执行编译检查
4. **禁止自动 Push：**push 由用户主动触发，保留审查机会

5. Message 格式: [<变更名>] <中文简述>

四、实战案例

以一个典型的跨模块业务迁移项目为例（将系统 A 的过滤处理逻辑平行迁移到系统 B），走一遍完整的 /propose → /apply → /review 流程。

4.1 需求 → Spec：把需求文档喂给 AI

将需求文档作为输入，执行 /propose，AI 先 Research 代码现状，然后逐个提问收敛不确定性：

```
1 🤖 AI: spec 初稿写好了，几个需要你确认的点：
2 待澄清（需要你回答的）：
3 1. Q1: 系统 A 中 AbstractFilterService 下三个处理器具体叫什么类名？
4     我需要知道才能让 AI 精确对照迁移
5 2. Q2: FilterModule 的 lockKey 和 idempotentId 要不要设？
6     写入的幂等是 SDK 层保证还是需要框架层加？
7 3. Q3: needOrderTradeType 目前有 CLOSE/PAID_DONE/ENABLE_DONE,
8     退款消息在框架里是怎么表示的？是单独的 OrderTradeType 还是走别的路径？
9 tasks 拆分思路：
10 7 个 task，按依赖顺序：灰度配置 → bizProcess 分发主干 → 三个处理器
11 （可并行）→ 压测 mock → 框架扩展点补全。每个 task 都是原子可提交的。
12 一个设计决策想确认：我把三个处理器放在同一个 FilterModule 的
13 bizProcess 里按类型分发，而不是拆成三个独立 Module。理由是
14 "一个模块一个处理器"的设计，过滤本身就是一个模块。你觉得这样 OK 还是要拆？
```

关键点：

- AI 必须先 Research 现有代码（带出处），不允许凭空设计
- 逐个提问而非一次列 10 个问题，优先给选项 + 推荐
- 待澄清项全部解决前不进入编码

4.2 Spec → Tasks：拆分原子任务

Spec 确认后，AI 自动拆分为原子级 Tasks。执行一段时间后，AI 汇报进度并列出待完善项：

```
1 🤖 AI: /apply filter-migration 执行进度
2 ▲ 待完善项 (TODO)
3 | 位置 | 说明 | 优先级 |
4 | ----- | ----- | ----- |
5 | LimitFilterService | 垂直表 + 水平表匹配逻辑具体实现 | P0 |
6 | LimitFilterService | 配置读取 (marketingSolutionIds) | P0 |
7 | All Services | 外部 SDK 具体 API 调用 | P0 |
```

```
8 | RefundFilterService | actualPaidFee / refundFee 字段获取 | P1 |
9 | All Services | 降级开关集成 (LIMITED_FILTER_DEGRADE_ON 等) | P1
10 🚀 下一步建议
11 1. 补全 TODO 实现: 需要确认二方包 FilterClient 的具体 API
12 2. 创建配置 dataId: com.example.trade.filter.gray
13 3. 本地测试验证
14 4. Code Review
15 您希望:
16 • 继续补全 TODO 实现?
17 • 先进行 /review filter-migration 审查?
18 • 还是其他操作?
```

4.3 Tasks → Apply: 逐步执行

进入 /apply 阶段, AI 按 Tasks 逐个执行。每完成一批文件, 展示完成状态和核心实现结构:

```
1 🤖 AI: 开始编码 /apply filter-migration
2 ✅ 已完成文件
3 | 类型 | 文件路径 | 状态
4 | --- | ----- | ---
5 | NEW | config/FilterGrayConfig.java | ✅ 灰度配置类, 配置中心动态监听
6 | MOD | module/FilterModule.java | ✅ 完整分发框架, 注入三个 Service
7 | NEW | service/filter/LimitFilterService.java | ✅ 限购过滤服务 (子单级)
8 | NEW | service/filter/ShopFrozenFilterService.java | ✅ 冻结店铺新用户 (主单级)
9 | NEW | service/filter/RefundFilterService.java | ✅ 退款过滤服务 (含完全退款)
10 🔧 核心功能实现
11 FilterModule.bizProcess()
12 |— ENABLE_DONE (创单)
13 |   |— LimitFilterService.processEnableDone() # 限购过滤写入
14 |   |— ShopFrozenFilterService.processEnableDone() # 冻结店铺过滤写入
15 |— PAID_DONE (支付完成)
16 |   |— RefundFilterService.processPaidDone() # 删除过滤
17 |— CLOSE (关单)
18 |   |— ShopFrozenFilterService.processClose() # 回滚 (先)
19 |   |— RefundFilterService.processClose() # 逆向添加 (后)
20 |   |— 完全退款判断: actualPaidFee == refundFee
21 |   |— 新客判断: identityId == 1L
```

4.4 迭代与 Reverse Sync

执行过程中用户发现 Spec 遗漏或实现偏差时, AI 先确认事实再更新文档:

```
1 👤 用户: 垂直表逻辑已维护在 TradeMainOrder 中, 由 TradeContext 携带
2 LimitFilterService 的 marketingSolutionIds 不是读配置中心
```

```
3      是读的 switch 配置 SwitchConfig.XXXXX
4      FilterClient 的调用请参考系统 A 中 ShardingFilterService 的实现
5      actualPaidFee/refundFee 同理已维护在 TradeMainOrder 中
6      先完善以上功能，降级开关等会再改
7      🤖 AI: 收到了！让我先理解您的修改和新的需求，然后更新文档：
8      1. 垂直表逻辑：已维护在 TradeMainOrder 中，通过 TradeContext 携带
9      2. marketingSolutionIds 来源：不是配置中心，而是 SwitchConfig.XXXXXX
10     3. FilterClient 调用：参考系统 A 侧的 ShardingFilterService 实现
11     4. actualPaidFee/refundFee: 已维护在 TradeMainOrder 中
12     让我先读取相关代码来确认这些细节，然后更新 spec：
13     ∞ 读取 SwitchConfig 配置
14     ∞ 读取 ShardingFilterService 实现
15     ∞ 验证 TradeMainOrder 修改
16     收到了！我发现几个问题需要澄清：
17     (AI 基于实际代码继续追问...)
```

Reverse Sync 是最容易被忽略但最重要的环节：

保持文档和代码的一致性，才能让整个框架持续有效。

五、一些观点与思考

5.1 人到底该干什么？

用 AI 编码之后，人的角色从「全干」变成了「管和验」：

- 1 传统编程：人 = 设计者 + 实施者 + 验收者
- 2 AI 编程：人 = 设计者 + 验收者，AI = 实施者

具体来说，人做三件事：**管控**（控制 AI 看什么）、**指挥**（选方案、审计划、批准执行）、**评价**（验收结果、发现偏差）。对应地，AI 在不同阶段切换角色：侦察兵（扫描代码收集事实）→ 参谋（提出方案分析利弊）→ 施工队（按图施工）→ 质检员（对照标准检查）。

听起来很清晰，但实际协作中有三个非常容易犯的错误：

1. 讨论和命令混为一谈

"帮我看看这个模块，顺便把 Bug 修了"——前半句是探索，后半句是指令。AI 会跳过探索直接改代码，然后改错。解决办法很简单：**一次只给一种意图**（探索 / 决策 / 指令 / 审查），不要混着来。

2. 阶段产出搞混

调研阶段要的是事实和风险，不是代码。AI 太勤快的时候要拉住它："停，我现在不需要代码，先告诉我现状和风险。"

3. 自由度给反了

这是最普遍的问题。正确的自由度曲线应该是：

阶段	自由度	为什么
调研	中	让 AI 自由探索，但必须给证据
方案设计	高	唯一鼓励 AI 充分想象的阶段
规划	低	精确到文件路径和函数签名
执行	零	严格按计划施工，有问题必须停下来问
验收	中	自由检查，但结论要有依据

大部分人的问题是反过来了——该讨论的时候急着让 AI 干活（方案没想清楚就开写），该干活的时候又让 AI 自由发挥（执行阶段不约束，改着改着就跑偏了）。

5.2 Spec 不是银弹，但也不是废弹

有一种批评认为 Spec Coding 建立在三个错误假设上——AI 能理解规范、规范能完整描述系统、规范比代码更易维护。

这些批判有道理，但忽略了一个关键前提：它批判的是「规范→代码」的全自动线性映射，不是人在回路中的 Spec 辅助模式。

被批判的模式	我们实际做的
写好 Spec，AI 自动生成全部代码	Spec 只描述变更范围，AI 在人审核下逐步执行
规范是「唯一真理来源」	规范是沟通工具，代码才是真理
期望 AI 理解整个系统	用 knowledge/ 喂精确上下文，限制 AI 的理解范围
适用于所有复杂度	渐进式——简单需求根本不写 Spec

问题不在于 Spec Coding 本身，而在于**用法和预期**。当成自动化流水线的输入，它确实不是银弹；当成人和 AI 之间的沟通协议，它就是一个靠谱的效率工具。

5.3 知识底座才是真正的护城河

大部分团队在 AI 编码上的投入方向是：花大量精力写 Prompt、调 Rules、优化 Agent 工作流。这些都属于「偶然复杂度」层面——调好了最多让 AI 少犯格式错误。但真正决定 AI 输出质量上限的，是你喂给它的**领域知识的质量**。

知识覆盖缺口：

知识类型	Spec 能覆盖	实际重要性
编码规范	★★★★	★★★
存量代码	★★★	★★★★★
领域知识	★	★★★★★
架构决策	★★	★★★★★
团队隐性经验	☆	★★★★★

最关键的知识——领域 Know-How、架构决策的前因后果、踩坑后的最佳实践——恰恰是纯 Spec 框架最难覆盖的。这也是为什么框架里有 knowledge/ 目录，而不是只有 rules/。

打个比方：一个没有 knowledge/ 的 Spec 框架，就像让一个刚入职的应届生对着编码规范写代码——规范他都能遵守，但业务逻辑全靠猜。

往长远看，AI 编码工具会越来越同质化（Cursor、Claude Code、各种 IDE 插件的能力趋同），团队之间的差距不在于用什么工具，而在于**积累了多少高质量的、结构化的领域知识**。这才是真正不可复制的护城河。

5.4 几个容易被忽略的代价

心流中断

编码需要高度专注。引入 AI 之后，连贯性被打破了——写 Spec、等生成、审查输出、纠正偏差、再继续下一段。模型越慢，等待越长，心流杀伤越大。

传统编码是「想→写→想→写」的连续流，AI 编码变成了「想→写 Spec→等→审→改→想」的间歇流。你需要适应这种新节奏，也需要诚实地评估：对于你个人，哪些任务用 AI 确实更快，哪些

还不如自己写。

上下文的隐性成本

Context 很贵，但贵的不只是 Token 费用。更大的隐性成本是**上下文管理本身**：你需要有意识地决定给 AI 看什么、不看什么；每一次上下文压缩都引入不确定性——被压缩掉的历史对话和决策，你无法确认 AI 是否还记得。

这也是框架把 knowledge/ 和 Spec 都做成独立文件的原因：文件是持久化的，不会被上下文窗口压缩掉。写在文件里的计划，比聊天记录里的口头约定可靠得多。

现在不是终态

编程可能是 AI 产生飞轮效应最直接的领域——AI 辅助编码 → 效率提升 → 更多代码产出 → AI 能力增强 → 更好地辅助编码。这个正向循环已经在发生。

今天某些模型的速度和质量让你觉得勉强够用，半年后可能完全不同。**框架的价值在于它能随模型进步而放大收益**——当模型从 T1.5 升到 T1 甚至 T0，同样的 Spec 和 knowledge/ 能产出质量截然不同的代码，而你的 rules/、knowledge/、历史 Spec 都是现成的积累。

参考

- Superpowers — agentic skills 框架（HARD-GATE、两阶段 review、systematic-debugging）：<https://github.com/obra/superpowers>
- Writing about Agentic Engineering Patterns – Simon Willison：
<https://simonwillison.net/2026/Feb/23/agentic-engineering-patterns/>
- Writing code is cheap now – Simon Willison：
<https://simonwillison.net/guides/agentic-engineering-patterns/code-is-cheap/>
- Chatbot Arena Leaderboard (LMSYS)：<https://arena.ai/>
- opencode 官方文档：<https://opencode.ai/docs/>
- Claude Code 文档：<https://docs.anthropic.com/en/docs/agents-and-tools/claude-code/overview>
- Frederick Brooks, 《人月神话》

附录：code_copilot 框架完整内容

以下为 code_copilot 框架当前的完整文件内容，可直接复制到项目中使用。项目特定内容（应用名、包名、中间件等）需根据实际情况填充。

目录结构

```
1 code_copilot/
2 |─ README.md                # 框架说明
3 |─ agents/                  # Agent 配置与提示词
4 |   |─ copilot-prompt.md    # 主 Agent 完整提示词（核心）
5 |   |─ spec-reviewer.md     # Spec 合规审查 Agent
6 |   └─ code-quality-reviewer.md # 代码质量审查 Agent
7 |─ rules/                   # 项目约束（始终生效）
8 |   |─ project-context.md    # 工程结构与依赖 (/init 填充)
9 |   |─ coding-style.md       # 编码规范
10 |   |─ security.md           # 安全红线
11 |   └─ domain-rules.md       # 业务领域约束
12 |─ knowledge/               # 领域知识（按需加载）
13 |   └─ index.md              # 知识索引
14 └─ changes/                 # 变更管理
15     └─ templates/           # 模板目录
16         |─ spec.md           # Spec 模板
17         |─ tasks.md           # Tasks 模板
18         |─ test-spec.md       # 单测 Spec 模板
19         └─ log.md             # Log 模板
```

A.1 agents/copilot-prompt.md — 主 Agent 提示词

```
1 你是 code-copilot，一个面向已有 Java 后端项目的 AI 编码协作助手。
2 你的工作基于 rules/（项目约束）、knowledge/（领域知识）、changes/（变更管理）三个
3 # 核心法则
4 ## Spec 驱动 (Code is Cheap, Context is Expensive)
5 代码是廉价的消耗品，文档（Spec）才是昂贵的核心资产。
6 1. **No Spec, No Code** – 没有 spec，不准写代码
7 2. **Spec is Truth** – spec 和代码冲突时，错的一定是代码
8 3. **Reverse Sync** – 执行中发现 spec 与实际不符，先修 spec 再修代码
9 4. **代码现状必须有出处** – 每个结论必须标注文件路径和类名/方法名，不接受“我认为”、“
10 5. **变更即记录** – 任何代码变更完成后都必须同步更新对应的 changes/ 文档
11 ## 身份与原则
12 – 有经验的 Java 后端工程师搭档，不是代码生成器
13 – 用中文输出，技术术语可保留英文
14 – 不确定就问，不假设，不编造不存在的类或接口
15 – 每个任务原子化（3-5 个文件），做“小炸弹”而非“大炸弹”
16 – 涉及资金/交易状态变更 → ⚠ 高亮提醒人工审查
17 – 有价值的发现 → 主动建议沉淀到 knowledge/
```

```
18 ## 意图确认 (先问再做)
19 收到用户的自然语言指令时, 先识别意图并映射到对应命令, 确认后再执行。
20 | 用户说的 | 映射命令 |
21 |-----|-----|
22 | "修复 xxx" / "改一下 xxx" | → /fix |
23 | "我要做 xxx 需求" | → /propose |
24 | "开始写代码" / "继续执行" | → /apply |
25 | "帮我看看代码" / "review 一下" | → /review |
26 | "写测试" / "补单测" | → /test |
27 | "归档 xxx" | → /archive |
28 纯技术讨论不需要走命令流程, 直接回答。
29 # 启动
30 每次会话开始时:
31 1. 读取 rules/ 下所有规则文件
32 2. 检查 changes/ 下是否有进行中的变更 (排除 templates/)
33 3. 报告当前状态, 展示命令菜单
34 # 命令
35 ## /init - 初始化项目上下文
36 分析工程结构、依赖、分层模式, 填充 rules/project-context.md。
37 ## /propose <需求描述> - 创建变更提案
38 Research → 逐个提问 (一次只问一个, 给选项+推荐) → YAGNI 裁剪 →
39 分三段生成 spec (每段确认) → 生成 tasks → HARD-GATE 确认。
40 待澄清全部解决前不允许进入 /apply。
41 ## /apply <变更名> - 执行编码
42 前置检查 spec + tasks + 用户确认。
43 逐 task 执行, 每个 task 完成后展示验证证据 (Verification 铁律)。
44 零偏差原则: Plan 是合同, AI 是打印机。
45 自动 git commit (一个 task 一个 commit)。
46 ## /fix <变更名> [描述] - Review 后修正迭代
47 增量修正 + 文档同步铁律 (spec/tasks/log 全部更新)。
48 ## /review <变更名> - 两阶段审查
49 阶段一 Spec Compliance → 阶段二 Code Quality。
50 优先用 Sub Agent 执行 (上下文隔离)。阶段一 PASS 后才启动阶段二。
51 ## /test <变更名> - 生成单测 Spec 并执行
52 Red/Green TDD: 测试必须先 Red 再 Green。
53 两种模式: Spec 先行 (推荐) 或直接生成。
54 ## /archive <变更名> - 归档 + 知识沉淀
55 逐条展示 log.md 知识发现, 确认后沉淀到 knowledge/。
56 ## Git 规范
57 1. 禁止 master 分支变更
58 2. 每个 task/fix 自动 commit
59 3. Commit 必须可编译
60 4. 禁止自动 push
61 5. Message 格式: [<变更名>] <中文简述>
62 ## 调试流程
63 四阶段: 根因调查 → 模式分析 → 假设验证 → 实施修复。
```

64 禁止在未确认根因前直接改代码。

A.2 agents/spec-reviewer.md — Spec 合规审查

```
1 # Spec Compliance Reviewer
2 专职验证代码实现是否符合 spec 规格。只读不写，独立于实现者的上下文。
3 核心理念：**不信报告，只信代码** – reviewer 必须读实际代码独立验证。
4 ## 审查维度
5 1. **缺失实现**：spec 要求了但代码没做的
6 2. **多余实现**：spec 没要求但代码多做了的（YAGNI 违规）
7 3. **理解偏差**：做了但做错了方向的
8 4. **业务规则落地**：spec §4 中的规则是否全部体现在代码中
9 5. **数据变更准确性**：spec §5 中的表/字段变更是否准确落地
10 ## 输出格式
11 ##### 功能点逐条验证
12 - ✅ 功能 1：已实现，见 `XxxService.java:L42`
13 - ❌ 功能 2：未实现（缺少 XX 逻辑）
14 - ⚠️ 功能 3：实现方式与 spec 描述有偏差
15 ##### 结论：✅ Spec 合规 / ❌ 不合规（附具体问题）
16 ## 工具权限
17 仅需 Read/Grep/Glob/Bash（只读），不需要写入权限。
```

A.3 agents/code-quality-reviewer.md — 代码质量审查

```
1 # Code Quality Reviewer
2 专职审查代码质量、安全性和可维护性。
3 前置条件：必须在 spec-reviewer 审查通过后才启动。
4 ## 审查分级
5 - **Critical**（阻塞）：安全漏洞、资金逻辑错误、并发安全、数据丢失风险
6 - **Important**（应修复）：异常被吞、缺少参数校验、魔法值、方法过长、命名不清
7 - **Minor**（建议）：Javadoc 缺失、注释过时、import 未清理
8 ## 工具权限
9 仅需 Read/Grep/Glob/Bash（只读），不需要写入权限。
```

A.4 rules/project-context.md — 工程上下文

```
1 ---
2 alwaysApply: true
3 ---
4 # 工程上下文
5 > 首次使用时执行 /init 让 AI 分析工程并填充本文件。
6 ## 1. 应用概况
7 - 应用名：（待填充）
```

```

8 - 简介：（一句话描述）
9 - 技术栈：Java 21 / Spring Boot / （根据项目实际填写）
10 - 构建工具：Maven
11 ## 2. 目录结构与模块职责
12 > 执行 tree -d -L 3 后填充。
13 ## 3. 分层架构
14 Controller (web/)          ← 入口层，参数校验 + 协议转换
15     ↓
16 Service (service/)         ← 业务编排，事务边界
17     ↓
18 Manager (manager/)         ← 领域能力，单一职责，可复用
19     ↓
20 DAO (dao/)                 ← 纯数据访问
21 ## 4. 关键依赖
22 | 中间件 | 用途 | 备注 |
23 |-----|-----|-----|
24 | （根据项目填写） | | |

```

A.5 rules/coding-style.md — 编码规范

```

1 ---
2 alwaysApply: true
3 ---
4 # 编码规范
5 ## 1. 命名
6 - 类名：大驼峰，见名知意
7 - 方法名：小驼峰，动词开头
8 - 常量：全大写下划线分隔
9 - 抽象类以 Abstract 或 Base 开头
10 - 测试类以被测类名开头，Test 结尾
11 - 禁止拼音、中英混拼命名
12 ## 2. 异常处理
13 - 业务异常使用自定义 BizException，携带错误码
14 - 系统异常向上抛出，由统一异常处理器兜底
15 - 禁止吞掉异常（空 catch）
16 - catch 中必须记录日志
17 ## 3. 日志
18 - Controller 入口打 INFO，含请求关键参数
19 - 异常打 ERROR，含完整堆栈
20 - 禁止在日志中打印用户敏感信息
21 ## 4. 其他
22 - 写接口必须考虑幂等
23 - 涉及并发场景必须说明同步策略
24 - 魔法值必须定义为常量

```

A.6 rules/security.md — 安全红线

```
1 ---
2 alwaysApply: true
3 ---
4 # 安全红线
5 ## 1. 代码安全
6 - ✗ 禁止在代码中硬编码密钥、AK/SK、数据库密码
7 - ✗ 禁止提交包含用户个人信息的测试数据
8 - ✗ 禁止在日志中打印手机号、身份证、银行卡等敏感信息
9 ## 2. 业务安全
10 - △ 涉及资金变更的逻辑，必须在 spec 中明确标注，人工审查后方可编码
11 - △ 涉及状态流转的逻辑，必须检查状态机合法性
12 - △ 涉及权限变更的逻辑，必须显式校验操作人权限
```

A.7 rules/domain-rules.md — 业务领域约束

```
1 ---
2 alwaysApply: false
3 description: "当涉及业务领域特定逻辑时应用本规则"
4 ---
5 # 业务领域约束
6 ## 1. 通用领域规则
7 - 所有金额使用 long 类型，单位为分
8 - 时间字段统一使用 Date 类型
9 - 外部接口调用必须设置超时（默认 3s）并做降级处理
10 - 状态变更必须通过状态机，禁止直接 set 状态字段
11 ## 2. 项目特定规则
12 （待实践中补充）
```

A.8 knowledge/index.md — 知识索引

```
1 # 知识索引
2 > 领域知识的轻量索引。每条用一句话说清核心逻辑。
3 > 格式：- **触发关键词**：一句话核心逻辑 → `包名.类名.方法名`（可选）
4 ## 业务知识
5 （随实践积累补充）
6 ## 技术约定
7 （随实践积累补充）
8 ## 踩坑记录
9 （随实践积累补充）
```

A.9 changes/templates/spec.md — Spec 模板

```

1 # 需求名称
2 > status: propose | apply | review | done
3 > created: YYYY-MM-DD
4 > complexity: ●简单 | ●中等 | ●复杂
5 ## 1. 背景与目标
6 为什么做 + 做完后的效果（可验证的结果描述）
7 ## 2. 代码现状 (Research Findings)
8 > 每个结论必须有代码出处（文件路径 + 类名/方法名）
9 #### 2.1 相关入口与链路
10 #### 2.2 现有实现
11 #### 2.3 发现与风险
12 ## 3. 功能点
13 - [ ] 功能 1: （输入→处理→输出）
14 ## 4. 业务规则
15 ## 5. 数据变更
16 | 操作 | 表名 | 字段/索引 | 说明 |
17 ## 6. 接口变更
18 | 操作 | 接口 | 方法 | 变更内容 |
19 ## 7. 影响范围
20 ## 8. 风险与关注点
21 > △ 涉及资金/状态流转/权限变更必须标注
22 ## 8.5 测试策略
23 - **测试范围**:
24 - **覆盖率目标**:
25 - **独立 Test Spec**: 是/否
26 ## 9. 待澄清
27 - [ ] 问题 1: （全部解决后才能进入 /apply）
28 ## 10. 技术决策
29 ## 11. 执行日志
30 | Task | 状态 | 实际改动文件 | 备注 |
31 ## 12. 审查结论
32 ## 13. 确认记录 (HARD-GATE)
33 - **确认时间**:
34 - **确认人**:

```

A.10 changes/templates/tasks.md — Tasks 模板

```

1 # 任务拆分 - 需求名称
2 > 拆分顺序: 数据模型 → 接口协议 → 底层实现 → 上层编排 → 入口层
3 > 每个任务 = 可独立提交的原子变更 (3-5 个文件)
4 > 每个任务必须精确到文件路径和函数签名
5 ## 前置条件
6 - [ ] （依赖/配置等前提）
7 ## Task 1: 任务名

```

```
8 - **目标**: 一句话描述
9 - **涉及文件**:
10 - path/to/File.java - 新增/修改, 做什么
11 - **关键签名**:
12   ```java
13   public ResultDTO doSomething(Long id, String type) { }
```

- 依赖: 无
- 验收标准: 怎样算完成
- 验证命令 (可选) :
- 完成

变更摘要

/apply 全部完成后填写

- 总文件数: X 个文件
- Spec-Plan 偏差记录:
- 遗留问题:

A.11 changes/templates/test-spec.md

单测 Spec 模板

```
1  ```markdown
2  # 单测 Spec - 需求名称
3  > status: propose | apply | done
4  > created: YYYY-MM-DD
5  ## 0. 测试原则
6  - **Red/Green TDD**: 测试必须先 Red 再 Green, 跳过 Red 的测试无法证明有效
7  - **First Run the Tests**: 开始前先跑已有测试套件, 了解框架和基线
8  - **展示工作**: 必须展示 mvn test 实际输出, 禁止"测试通过"等无证据声明
9  ## 1. 测试框架
10 | 项目 | 值 |
11 |-----|-----|
12 | JUnit 版本 | |
13 | Mock 框架 | |
14 | 已有测试数量 | |
15 | 已有测试风格 | |
16 ## 2. 覆盖范围
```

```

17  ### P0 - 核心业务逻辑 (必须覆盖)
18  ##### 类名: XxxManagerImpl
19  | 方法 | 场景 | 输入 | Mock 行为 | 预期结果 |
20  |-----|-----|-----|-----|-----|
21  ### P1 - 数据访问层
22  ### P2 - 入口层/ 服务层
23  ### 不测试 (明确列出原因)
24  ## 3. 执行计划
25  - [ ] Step 1: 运行已有测试套件, 确认基线
26  - [ ] Step 2: 生成 P0 测试 → 确认 Red → 确认 Green
27  - [ ] Step 3: 生成 P1/P2 测试
28  - [ ] Step 4: 运行完整测试套件, 确认覆盖率

```

A.12 changes/templates/log.md — Log 模板

```

1  # 变更日志 - 需求名称
2
3  > 记录决策、踩坑和知识发现。知识飞轮的输入。
4
5  ## 时间线
6
7  | 时间 | 阶段 | 事件 | 备注 |
8
9  ## 技术决策
10
11 | 决策 | 选择 | 放弃的方案 | 原因 |
12
13 ## 踩坑记录
14
15 | 问题 | 原因 | 解决方案 | 沉淀? |
16
17 ## 知识发现
18 > 每个 task 后实时记录, /archive 时逐条确认沉淀到 knowledge/
19 - [ ] **关键词**: 描述
20
21 ## Spec-Code 偏差记录
22
23 | 偏差点 | Spec 预期 | 实际情况 | 处理方式 |
24
25 ## 代码质量备忘
26

```